

Lecture 22

Addition of two polynomial

- Adding terms having same variable and same exponent

$$4x^2 + 3x + 1$$

$$5x^3 + 4x^2 + 3x + 1$$

$$4x^2 + 3x + 1$$

$$5x^2 + 7x + 2$$

$$5x^2 + 7x + 2$$

$$10x^3 + 5x^2 + 7x + 2$$

$$9x^2 + 10x + 3$$

$$5x^3 + 9x^2 + 10x + 3$$

$$10x^3 + 9x^2 + 10x + 3$$

Algorithm

node structure

```
struct poly{  
    int coeff;  
    Int exp;  
    struct poly *next;  
};  
  
typedef struct poly node;
```

continue...

```
insert    head, tail, coeff, exp:

    if head == NULL or head -> exp < exp:

        insertAtHead(head, coeff, exp)

    else:

        insertAtTail(tail, coeff, exp)
```

continue...

```
polyAdd (head1, head2):  
    temp1 = head1  
    temp2 = head2  
    head = NULL  
    tail = NULL  
    while(temp1 != NULL && temp2 != NULL):  
        if temp1 -> exp == temp2 -> exp:  
            head = insert(head, tail, temp1->coeff + temp2->coeff, exp)  
        else if temp1 -> exp > temp2 -> exp:  
            head = insert(head, tail, temp1->coeff, temp1 -> exp)  
        else:  
            head = insert(head, tail, temp2 ->coeff, temp2 -> exp)
```

continue...

```
while(temp1 != NULL):
```

```
    head = insert(head, tail, temp1 -> coeff, temp1 -> exp);
```

```
    temp1 = temp1 -> next
```

```
while(temp2 != NULL):
```

```
    head = insert(head, tail, temp2 -> coeff, temp2 -> exp);
```

```
    temp2 = temp2 -> next
```

Multiplication of two polynomials

- Each term of polynomial 1 must be multiplied with each term of polynomial 2
- Multiplying each term means multiplying their coefficients and adding their exponents

Example

$$4x^2 + 5x + 3 \text{ ----- } 1$$

$$3x^3 + 6x \text{ ----- } 2$$

$$\text{Result} = (4*3)x^{2+3} + (4*6)x^{2+1} + (5*3)x^{1+3} + (5*6)x^{1+1} + (3*3)x^{0+3} + (3*6)x^{0+1}$$

Algorithm

```
polyMul(head1, head2):
```

```
    temp1 = head1
```

```
    temp2 = NULL
```

```
    newCoeff, newExp
```

```
    head = NULL, tail = NULL
```

```
    while(temp1 != NULL):
```

```
        temp2 = head2
```

```
        while(temp2 != NULL):
```

```
            newCoeff = temp1 -> coeff * temp2 -> coeff
```

```
            newExp = temp1 -> exp + temp2 -> exp
```

```
            head = insert(head, tail, newCoeff, newExp)
```

```
            temp2 = temp2 -> next
```

```
        temp1 = temp1 -> next
```